

Bit-identical transformer inference across ISAs, with a verifiable decode receipt, demonstrated on bare metal

Justin B. Thompson Aefinity AI Inc., Orange, Texas

2026-08-27

Abstract

We define CIS-1, a frozen integer-only semantics for the transformer decode path — every operation, attention included, in fixed-point integers under one rounding rule — and show it is checkable, not merely accurate. Given only the spec text and a public harness, two independent implementers reproduced the op-level digest on first try [A31]. The digest is bit-identical across four x86 microarchitecture/codegen paths [A25] and on aarch64 [A28], and a complete 64-token decode digest matches across both ISAs [A29]. A decode receipt — hashes of the model artifacts, the tokens, and a hash chain over every step's logit vector — verifies across the ISA boundary in public CI [A32], and is re-derived bit-for-bit by a UEFI unikernel booting with no OS, on two physical machines spanning AVX2 and scalar SSE2 [A33, A34]. The cost is microarchitecture-dependent and sign-flips: the integer path is 25% slower than scalar float on Broadwell-U and 4–14% faster on Goldmont Plus [A26]; at parity SIMD, the integer AVX2 kernel is 2.94× faster than the float AVX2 kernel [A27]. Quality cost, gated in advance, closed last: the complete all-integer forward pass on a 2-billion-parameter ternary model costs +0.1239% perplexity against float — 40× inside a preregistered +5% kill line [A35].

1. Introduction

Ask two machines to run "the same" quantized language model on the same prompt and you will usually get two different sets of logits. The weights are identical; the arithmetic is not. Parallel reductions sum in different orders, dequantization paths differ between kernels, normalization layers stay in floating point, and vendor libraries choose algorithms at runtime. The disagreements are small — a few units in the last place — and they are also decisive: greedy decoding is a chain of argmax operations, and one flipped comparison changes every token that follows. In practice, "reproducible" means "statistically similar," and it is hard to say exactly what a deployed floating-point model computed on a given input.

This paper makes reproducibility exact and then makes it *checkable*. We define **CIS-1**, a canonical integer semantics for transformer inference: a frozen specification in which every operation of the forward pass — the ternary matrix–vector product, activation quantization, RMS normalization, softmax, rotary position embedding, the MLP nonlinearity, and the final argmax — is an integer computation with pinned constants and a single normative rounding rule. A conforming implementation is free to use any instruction set, any vectorization, any parallelization; it is *not* free to produce a different bit. Conformance is not a matter of opinion or tolerance: it is a digest. An op-level self-test prints one 64-bit value, and a complete 64-token greedy decode of a reference model prints another. Either your implementation prints those values or it does not conform.

Because the semantics are exact, they support something floating-point inference cannot: a **decode receipt**. A receipt binds cryptographic hashes of the three model artifacts, the generated token ids, a digest over those tokens, and a hash chain over the complete integer logit vector at every decode step. Any conforming machine can replay the decode from source and verify the receipt bit-for-bit. The receipt is not a claim about a trusted platform; it is a claim about a computation, and it is checkable by anyone with the artifacts and the spec.

We demonstrate the following, each backed by an entry in the project's public research ledger with the raw log it was taken from (Table 3):

1. **The specification is implementable from its text.** Two implementers — language-model agents run under distinct personas, given the specification text and the public conformance harness, and denied access to the reference source — each wrote a from-scratch scalar implementation that reproduced the op-level digest on first execution (ledger A31). This is evidence of implementability from text, not a third-party audit (§8).

2. **One semantics, two instruction sets.** The op-level digest is identical on four distinct x86 microarchitecture/codegen paths — two bare-metal machines spanning AVX2 and SSE2-class hardware, a virtualized development host, and full-system emulation (A25) — and on an aarch64 Neoverse system (A28). A NEON kernel is bit-identical to the reference on real ARM silicon (A30), and a complete greedy decode produces the same token digest on x86-64 and aarch64 (A29). Both digests are standing continuous-integration gates: a future divergence fails the build.
3. **The receipt crosses the ISA boundary and the operating-system boundary.** A receipt minted on x86 verifies on aarch64 in public CI (A32). The same receipt is re-derived by a unikernel that boots from UEFI firmware with no operating system present, on two physical machines — one executing the AVX2 path, the other (which lacks AVX2) the scalar path (A33, A34).
4. **Determinism is not the expensive part.** Measured on physical hardware, the integer semantics cost 25% against scalar floating point on a Broadwell-U core and are 4–14% *faster* on a Goldmont Plus core (A26); at parity vector width the integer AVX2 kernel is 2.94× faster than the hand-written floating-point AVX2 kernel it replaces (A27). Earlier and larger "cost of determinism" figures, including this project's own, decompose into the cost of missing SIMD, not of integer semantics.
5. **The quality cost is small and was gated in advance.** Against a preregistered +5% perplexity kill line, the all-integer forward pass costs +0.06% on a 384-hidden reference model (A20), the integer-dominant hybrid path (attention still in floating point) costs +0.7408% on a 2-billion-parameter ternary model (A21), and the complete all-integer forward pass on the same 2B model costs +0.1239% — closer to float than the hybrid figure, and 40× inside the kill line (A35). The first all-integer run at this scale panicked instead: a NORMQ precondition rejected an out-of-range residual, tracing to a gap in spec §5.10, where the ACT-I MLP nonlinearity was the one op not carrying the spec §5.6 per-vector block exponent that the hybrid boundary already applied. The fix — an RNE-rounded block exponent on the ACT-I output — is ratified as spec erratum v1.0.3; both pinned conformance digests are unchanged.
6. **The token-level identity claim now reaches production scale, on one ISA.** A complete 64-token greedy decode of the 2-billion-parameter BitNet-2B model, run in the FullInt configuration, prints one digest, reproduced identically across two sequential runs on x86-64: `CIS_DECODE digest=cab11400d737ac4a prompt_toks=4 gen_toks=64 mode=fullint`, and the generated text is coherent English (A36). This is the x86 anchor for the BitNet-2B cross-ISA leg — the 2B counterpart of A29 — but the aarch64 leg has not yet been run.
7. **A third, independent implementation verifies the receipt without the engine.** `cis-verify`, a standalone crate with zero external runtime dependencies and no dependency on `aegis-core`, reproduces both pinned conformance digests and verifies the golden receipt — all six checks (parse, three artifact hashes, prompt tokenization, the 64-step token sequence, the cis-digest, and the witness chain) — in about 1.4 seconds, with tamper tests that fail by naming the corrupted field (A37). Honest scope: this was an LM-agent transcription of the spec with the reference source visible, not a clean-room audit (§8).

We are equally explicit about what is *not* shown (§8): the clean-room implementers were language-model agents rather than third-party engineers; the physical-machine attribution of one boot log rests on operator witness because the verifier prints no CPU identifier; the 2B-parameter perplexity figures (hybrid and all-integer) use a pruned vocabulary and a short window and are not comparable to published numbers or to this project's own longer-window anchors; and no token-level throughput figure for the integer path has yet been measured. The project publishes negative results and retractions in the same ledger as its claims, and a standing falsification bounty invites anyone to find a machine on which the digests diverge.

The contribution, then, is not a faster kernel or a smaller model. It is a way to say, and to prove, exactly what a model computed — on hardware you do not have to trust, without an operating system you would otherwise have to audit.

2. Background and related work

Ternary-weight and integer-only LLMs. BitNet b1.58 quantizes every weight to $\{-1, 0, +1\}$ (1.58 bits/parameter) and trains directly at that precision rather than quantizing a float-trained model post hoc; Ma et al. report perplexity and end-task parity with FP16 LLaMA baselines at matched size and training tokens, alongside large memory, latency, and energy reductions (Ma et al., "The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits," arXiv:2402.17764, Feb 2024). Microsoft's follow-up technical report open-sources a 2B-parameter, 4T-token BitNet b1.58 checkpoint with public weights and CPU/GPU inference code (Ma et al., "BitNet b1.58 2B4T Technical Report," arXiv:2504.12285, Apr 2025). A companion systems paper describes CPU kernels for ternary inference and reports CPU speedups of

2.37–6.17× on x86 and 1.37–5.07× on ARM over unoptimized baselines (Wang et al., "1-bit AI Infra: Part 1.1, Fast and Lossless BitNet b1.58 Inference on CPUs," arXiv:2410.16144, Oct 2024); a related report, `bitnet.cpp`, targets edge deployment of ternary models (arXiv:2502.11880). None of this line of work specifies a bit-exact numerical contract for the non-matmul path (normalization, softmax, rotary embedding, requantization): the object released is a model and a fast kernel, not a semantics that a second, independent implementation is obligated to reproduce bit-for-bit. That gap — fast ternary inference exists; a frozen, checkable numerical contract for it does not — is the space CIS-1 occupies.

Non-determinism and reproducibility in DL inference. Floating-point addition is not associative, so reduction order — which varies with thread scheduling, tiling, batch size, and kernel choice — changes results at the bit level; Shanmugavelu et al. quantify this across HPC and deep-learning workloads and evaluate deterministic-summation and hardware-level mitigations, including PyTorch's determinism flags (arXiv:2408.05148, Aug 2024, rev. Oct 2024). Horace He's widely-cited analysis for Thinking Machines Lab argues the dominant real-world cause of LLM-inference nondeterminism is not concurrency per se but *batch-size variance*: individual GPU kernels (matmul, RMSNorm, attention) are themselves deterministic for a fixed batch shape but change their reduction strategy — and therefore their bit-level output — as batch size changes; the proposed fix is batch-invariant kernels, measured at roughly 20% overhead for matmul/RMSNorm and up to ~2× unoptimized for attention (He, "Defeating Nondeterminism in LLM Inference," Thinking Machines Lab blog, Sep 2025). DiFR frames the adjacent problem of verifying a provider's inference under inherent nondeterminism, comparing generated tokens or compressed activation fingerprints against a trusted reference to catch, e.g., undisclosed quantization (Karvonen et al., arXiv:2511.20621, Nov 2025). CIS-1 targets a different point in this space: rather than tolerating float nondeterminism and verifying *around* it statistically, it removes floats from the decode path so that reduction order provably cannot change the result, making the DiFR-style tolerance machinery unnecessary for a conforming implementation.

Integer and fixed-point transformer inference. I-BERT performs end-to-end BERT inference in integer-only arithmetic, replacing GELU, Softmax, and LayerNorm with polynomial/lookup integer approximations (Kim et al., arXiv:2101.01321, Jan 2021, rev. Jun 2021). I-ViT extends this to Vision Transformers with Shiftmax and ShiftGELU, reporting INT8 accuracy comparable to full precision (Li and Gu, arXiv:2207.01405, Jul 2022, rev. Aug 2023, ICCV 2023). A 2026 report proposes a clipped-linear softmax surrogate for int8 edge accelerators (Danopoulos et al., "Taming the Exponential," arXiv:2604.02292, Apr 2026) — UNVERIFIED beyond its abstract, not read in full. These works establish that individual nonlinear ops *can* be made integer-only; none states its integer approximations as a frozen, versioned specification with pinned rounding and a conformance digest, nor addresses RoPE. That distinction — approximation technique vs. normative, checkable semantics — is the same gap noted for BitNet above.

Verifiable inference. Platform (not computational) attestation is mature: TCG measured boot extends per-stage hashes into TPM PCRs, signed and reported to a verifier (Trusted Computing Group, "Overview of TCG Technologies for Device Identification and Attestation," v1.0 rev. 1.37, Feb 2024); NIST's SP 800-155 draft specifies BIOS/firmware integrity measurement guidelines for the same chain (NIST SP 800-155 (Initial Public Draft), Dec 2011). This attests *what code ran*, not what it computed. Cryptographic approaches instead prove computation: zkLLM proves correct LLM inference in zero-knowledge with a specialized attention argument, generating a full-inference proof for a 13B model in under 15 minutes with sub-200 kB proofs (Sun et al., arXiv:2404.16109, Apr 2024, ACM CCS 2024); a 2025–2026 survey catalogs the broader ZKML literature across training, testing, and inference (Peng et al., arXiv:2502.18535). A parallel line uses fully homomorphic encryption so a server computes on encrypted inputs without seeing them, e.g. Orion for FHE deep learning (arXiv:2311.03470); FHE and ZK proofs both target privacy/correctness against an untrusted *executor*, at heavy per-inference cost. CIS-1's decode receipt is a lighter-weight, complementary object: a commitment (SHA-256 chain) over an already-computed integer transcript, verified by re-derivation rather than by circuit proof, and — because the underlying arithmetic is order-invariant by construction — reproducible on different hardware without the prover/verifier asymmetry ZK and FHE approaches require. It attests *what was computed*, and is designed to compose with, not replace, platform attestation.

Conformance-by-digest as a design pattern. Cryptographic standards have long used exactly this shape: NIST's Cryptographic Algorithm Validation Program (CAVP) validates a black-box implementation by feeding it known inputs and checking outputs against known-correct answers (Known Answer Tests, Multi-block Message Tests, Monte Carlo Tests), most visibly for AES (NIST CAVP; AESAVS specification, `csrc.nist.gov`). Conformance is a match against a pinned test vector, not an audit of source code or implementation strategy — implementation freedom plus an exactness contract, verified by anyone who can run the vectors. CIS-1's two conformance digests (op-level and token-level) are this pattern applied to transformer inference: a specification document is normative, and any implementation — regardless of ISA, vector width, or language — either reproduces the pinned digest or does not conform.

Sources

- Ma et al., "The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits," arXiv:2402.17764 — <https://arxiv.org/abs/2402.17764>
- Ma et al., "BitNet b1.58 2B4T Technical Report," arXiv:2504.12285 — <https://arxiv.org/abs/2504.12285>
- Wang et al., "1-bit AI Infra: Part 1.1, Fast and Lossless BitNet b1.58 Inference on CPUs," arXiv:2410.16144 — <https://arxiv.org/abs/2410.16144>
- "Bitnet.cpp: Efficient Edge Inference for Ternary LLMs," arXiv:2502.11880 — <https://arxiv.org/pdf/2502.11880>
- Shanmugavelu et al., "Impacts of floating-point non-associativity on reproducibility for HPC and deep learning applications," arXiv:2408.05148 — <https://arxiv.org/abs/2408.05148>
- He, "Defeating Nondeterminism in LLM Inference," Thinking Machines Lab — <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>
- Karvonen et al., "DiFR: Inference Verification Despite Nondeterminism," arXiv:2511.20621 — <https://arxiv.org/abs/2511.20621>
- Kim et al., "I-BERT: Integer-only BERT Quantization," arXiv:2101.01321 — <https://arxiv.org/abs/2101.01321>
- Li and Gu, "I-ViT: Integer-only Quantization for Efficient Vision Transformer Inference," arXiv:2207.01405 — <https://arxiv.org/abs/2207.01405>
- Danopoulos et al., "Taming the Exponential: A Fast Softmax Surrogate for Integer-Native Edge Inference," arXiv:2604.02292 — <https://arxiv.org/abs/2604.02292> (UNVERIFIED beyond abstract)
- Trusted Computing Group, "Overview of TCG Technologies for Device Identification and Attestation," v1.0 rev. 1.37 — https://trustedcomputinggroup.org/wp-content/uploads/Overview-of-TCG-Technologies-for-Device-Identification-and-Attestation-Version-1.0-Revision-1.37_5Feb24-2.pdf (content summary drawn from search index only; full text not machine-readable via fetch — UNVERIFIED beyond title/version)
- NIST SP 800-155 (Initial Public Draft), "BIOS Integrity Measurement Guidelines" — <https://csrc.nist.gov/pubs/sp/800/155/ipd>
- Sun, Li, and Zhang, "zkLLM: Zero Knowledge Proofs for Large Language Models," arXiv:2404.16109 — <https://arxiv.org/abs/2404.16109>
- Peng et al., "A Survey of Zero-Knowledge Proof Based Verifiable Machine Learning," arXiv:2502.18535 — <https://arxiv.org/abs/2502.18535>
- "Orion: A Fully Homomorphic Encryption Framework for Deep Learning," arXiv:2311.03470 — <https://arxiv.org/abs/2311.03470>
- NIST, "Cryptographic Algorithm Validation Program (CAVP)" — <https://csrc.nist.gov/Projects/cryptographic-algorithm-validation-program>
- NIST/CSRC, "The Advanced Encryption Standard Algorithm Validation Suite (AESAVS)" — <https://csrc.nist.gov/csrc/media/projects/cryptographic-algorithm-validation-program/documents/aes/aesavs.pdf>

3. CIS-1: the semantics

CIS-1 rests on one axiom (spec §1): *every reduction in the decode path is a sum of integers whose worst case provably fits its accumulator*. Integer addition is associative and commutative, so a conforming implementation may reorder, vectorize, tile, or parallelize any computation — SSE2, AVX2, NEON, GPU, FPGA, a for-loop in Python — and MUST produce bit-identical results. The spec explicitly rejects the alternative of a canonical float summation order: pinning one loop structure would kill SIMD freedom and still break under FMA-vs-separate rounding. Determinism is therefore a property of the arithmetic itself, not a discipline the kernel author must maintain.

Rounding is fixed at one mode everywhere: round-half-even (RNE), with `rne(num/den)` for `den > 0` computed exactly in `i128` as the reference primitive (spec §2). The spec calls tie handling normative, not a detail, because the motivating token flips traced to near-ties decided by float rounding noise. Four transcendental constants are pinned as the RNE rounding of published 50-digit decimal expansions, and the integers themselves — not the decimals — are normative: `TWO_PI_Q62 = 28976077832308491370`, `PI_Q62 = 14488038916154245685`, `PI_2_Q62 = 7244019458077122842` (independently rounded, not derived from `PI_Q62`), and `LOG2E_Q32 = 6196328019`. Later pinned artifacts are verified the same way, by digest rather than by printing the value: the exp LUT (spec §5.7) carries FNV-1a 64 digest `0x66C2A0EEB8C2DC43`, golden-tested against an independent big-integer generator, and the RoPE table for the M7 shape `512×32` at base 10000.0 (spec §5.9) carries `0xD8345EBF01E990FA`.

Every object that crosses the decode path is assigned a type, a fixed-point grid, and a proven bound (spec §3). Quantized activations are symmetric `i8` in `-127..+127`, with `-128` explicitly forbidden. Dot-product accumulators are `i32`, valid for `dim_in ≤ [i32::MAX/127]`. The residual stream is `i64` in Q.20 with `|h| < 250` and vector length `n ≤ 8192`. q/k/v vectors are `i32` Q.16 with `|·| < 229` pre-rotation, enforced by a loud panic rather than

saturation. Attention scores are `i64` Q.24 with $|\text{score}| \leq 2^{59}$, accumulated in `i128`. Probabilities live on Q0.15 with Σp within $[T/2]$ of 2^{15} . `exp` LUT entries are Q0.31 (spec §5.7), RoPE tables are `i32` Q1.30 (spec §5.9), norm gains are `i32` Q.20 in the engine or `i16` in the reference op, and logits are `i64` from an exact integer dot. The spec states its headroom claim plainly: a ternary dot of length `L` over `i8` is bounded by $127 \cdot L$, and a sum of squares by $127^2 \cdot L$; violating a stated bound **MUST** be a loud failure, never a silent wrap or saturation.

Weight containers pack row-major, four weights per byte, low bit-pair first, with `dim_in` a multiple of 4 (spec §4). The two-bit codes are `00` = 0, `01` = +1, `10` = -1, and — ratified in this version — `11` decodes to 0 as well (defined-as-zero); a container holding `11` codes is conforming, and those positions simply contribute nothing to the dot product.

Section 5 defines twelve operations. **TMV** (spec §5.1), the ternary matrix–vector product, is exact in `i32` in any summation order; equally normative is its rejection surface — five preconditions (non-multiple-of-4 `dim_in`, an overflowing `dim_in`, and three length checks) that every implementation must reject identically regardless of ISA or internal blocking, with a rejected call writing no output. **QUANT-ACT** (spec §5.2) is per-token symmetric absmx quantization onto the `i8` grid, `q_i = rne(x_i · 127 / absmx)`, with the scale carried forward as the exact rational $127 / \text{absmx}$; the clamp that makes -128 unrepresentable is explicit. **REQUANT** (spec §5.3) applies a fixed-point multiplier $y = \text{clamp}(\text{rne}((\text{acc} \cdot M) / 2^{(31+S)}), -127, 127)$ with $M \in [2^{30}, 2^{31})$ and $S \leq 62$; the `i64`-input form used by RMSNORM-I is the identical arithmetic widened to `i128`. The offline generation of (M, S) from any rational is itself a normative integer procedure, including the exact S -selection rule and a renormalization case when RNE rounds M up to 2^{31} . **RMSNORM-I** (spec §5.4) specifies every intermediate grid — sum of squares in `i64`, an exact floor integer square root `t`, an inverse-RMS in Q2.30, and a final Q.20 output through **REQUANT** — because folding these roundings any other mathematically-equivalent way produces different `i8` outputs; this full procedure is a v1.0.1 erratum, since the original prose omitted the bit-determining grids. **NORMQ** (spec §5.5) is the fused engine form: the RMS factor cancels out of the `i8` codes algebraically and survives only in a carried exact rational scale. Spec §5.6 governs **container-boundary conversions** — bf16/f32-to-fixed conversions are exact RNE on the float bit pattern alone, never on accumulated values, and the block-exponent form `fix_f32_vec` **MUST** panic rather than saturate when its fractional width would go negative. Spec §5.7 defines the shared **exp machinery**: a parameter-free floor-isqrt chain generates a 1025-entry, strictly-decreasing Q0.31 LUT, looked up by a monotone interpolation. **SOFTMAX-I** (spec §5.8) works on the Q.24 score grid, subtracts the max exactly, and normalizes by exact RNE division rather than a Newton reciprocal, with the declared bound $|\Sigma p - 2^{15}| \leq [T/2]$. **ROPE-I** (spec §5.9) tables are not shipped but generated at load from $(\text{max_seq}, \text{head_dim}, \text{f32 bits of rope_theta})$ by a normative integer procedure — quadrant-reduced sin/cos via 9-term Taylor sums, RNE-requantized to Q1.30 and clamped to $\pm 2^{30}$. **ACT-I** (spec §5.10) elementwise ops work on the Q.20 grid: `relu²` is an exact squaring plus one more RNE requant, and `silu` evaluates one `exp_neg` call for $\sigma(g)$ in Q0.31 before two further RNE requants. Since the v1.0.3 erratum, the ACT-I output **MUST** also carry the spec §5.6 per-vector block exponent $(\text{shift} = \max(0, \text{bits}(\text{max}|v|) - 49))$ before entering **NORMQ**, identity at M7 ranges (spec §5.10, ledger A35). **ARGMAX** (spec §5.11) breaks exact-equality ties on the `i64` logits to the lowest index, making a tie a specified, reproducible event rather than rounding noise.

Spec §5.12 — pipeline grid assignments fix the grid at every stage of the forward pass:

Stage	Grid / accumulator
<code>q/k/v</code> (post-rescale)	<code>i32</code> , Q.16 (rotation growth $\leq \sqrt{2}$ stays inside <code>i32</code>)
Score dot	<code>i128</code> accumulator, scaled by $1/\sqrt{\text{head_dim}}$ in Q0.30, RNE'd to Q.24
V-mix ($\Sigma_t p_t \cdot v_t$)	exact <code>i64</code> , RNE onto the Q.20 residual grid
LM head	exact <code>i64</code> dot, <code>i8</code> -quantized hidden state $\times$ Q.20 embedding table

Section 6 states the exactness contract: an optimized kernel conforms only if it is byte-identical to the reference for *every* input, not merely inputs a well-behaved caller would produce. The worked hazard is -128: `x86 vpsignb` and `ARM vmulq_s8` both wrap -128×-1 within `i8` where the reference computes in `i32`, so both shipped kernels detect `i8::MIN` in their preparation pass and route the whole call to the reference — an unconditional guarantee, not one conditioned on the caller upholding an invariant. The rejection surface of spec §5.1 is likewise part of the op: whether a call is rejected must not depend on which CPU the binary landed on. The shipped vector kernels, `cis_avx2` and `cis_neon`, are informative rather than normative; their instruction-level claims are proven by exhaustive enumeration, and whole-kernel equality is checked by test suites — deliberately described as a test bar, not a proof.

Section 7 defines what a conforming engine computes: `CisMode::FullInt` runs embedding lookup, the residual stream, every norm, every activation quantization, all ternary matvecs, RoPE, attention scores, softmax, V-mix, the MLP elementwise ops, the LM head, and argmax such that no float value exists anywhere in the forward pass. Hybrid modes that keep attention or the MLP in `f32` carry no cross-ISA claim and sit outside conformance.

Conformance (spec §8) requires all three tiers to pass: Tier 1 op goldens, Tier 2 the operation-level digest, and Tier 3 the token-level digest. Tier 2 (`cis_selftest`) must print exactly:

```
CIS_SELFTEST digest=76985613c965f643 ALL_PASS=true
```

Tier 3 (`cis_decode` , prompt "Once upon a time" , 64 new tokens, greedy, EOS ignored, `FullInt`) must print exactly:

```
CIS_DECODE digest=67e8c0a96abc04e1 prompt_toks=4 gen_toks=64 mode=fullint
```

Source map - Axiom and rounding/constants paragraphs: spec §1, §2. - Data types, grids, headroom, and weight packing: spec §3, §4. - The twelve-operation survey: spec §5.1–§5.12. - Exactness contract and what a conforming engine computes: spec §6, §7. - Conformance digests: spec §8.

4. Implementability and cross-ISA identity

This section is correctness evidence only; no throughput or latency figure appears below (Rule A — identity legs carry no performance claim).

A four-implementation digest jury (A25). On 2026-08-02 the op-level self-test printed the identical digest, `CIS_SELFTEST digest=76985613c965f643 ALL_PASS=true`, on four distinct machines and codegen paths: the HP Stream (Celeron N4020, Gemini Lake, SSE2-class) running bare iron on minimal Linux, the Dell i5-5200U running bare iron across two cold boots, the `crosvm` dev host (i5-10210U), and QEMU/TCG full-system emulation. The ledger states its own scope exactly, and we repeat it rather than round it up: "Four distinct microarchitectures + codegen paths, one bit-exact integer semantics. Correctness leg only; no performance numbers taken from it" (A25).

The jury crosses its first ISA boundary (A28). On 2026-08-07 the same op-level digest, `76985613c965f643`, reproduced on a GitHub `ubuntu-24.04-arm` hosted runner (`uname -m=aarch64` asserted in-job, CPU part `0xd49`, Neoverse N2 class, Azure southcentralus). The reference semantics compiled untouched — `cis.rs` has zero imports, and `lib.rs` `cfg-gates` every x86-intrinsic module — and reproduced the digest on first execution. The standing CI gate `arm-digest.yml` now pins this digest on every push; a future mismatch fails the build, with the ledger's own instruction to preserve the log as a finding, not fix it into silence. Because this leg runs on a cloud runner, the machine is named as precisely as the platform allows; it stands as an identity artifact only — no timing is quoted or quotable from it (A28).

Table 2 collects the digest jury; every cell traces to A25 or A28.

Table 2 — the digest jury

Machine	Microarchitecture	Environment	Digest	Row
HP Stream (Celeron N4020)	Gemini Lake, SSE2-class	bare iron, minimal Linux	76985613c965f643	A25
Dell Inspiron 15 (i5-5200U)	—	bare iron (2 cold boots)	76985613c965f643	A25
<code>crosvm</code> dev host (i5-10210U)	—	<code>crosvm</code> (virtualized)	76985613c965f643	A25
QEMU/TCG	—	full-system emulation	76985613c965f643	A25
GitHub <code>ubuntu-24.04-arm</code> (Azure southcentralus)	Neoverse N2 class (CPU part <code>0xd49</code>)	hosted CI runner	76985613c965f643	A28

The specification is implementable from its text (A31). On 2026-08-08 two independent implementers — subagents under distinct personas — were given the v1.0.1 spec text as their sole source, with reference-source access forbidden and, as far as the harness can establish, not used. Each wrote a from-scratch scalar Rust implementation of the spec §5 operations (400 and 484 lines respectively, verified distinct by `diff/md5`). Both, linked against the verbatim-ported public conformance harness, printed exactly `CIS_SELFTEST digest=76985613c965f643 ALL_PASS=true` — all 14 sections, first run after their own cargo-check. The v1.0.1 spec §5.4 erratum to RMSNORM-I's Q2.30/2¹⁵ grids is what made this possible: both implementers hit RMSNORM-I's golden on first attempt from the corrected prose. The ledger states the honest scope plainly, and we do not soften it: the public conformance suite's golden vectors were visible to

the implementers, and both reports state the A8 goldens were needed to settle `from_ratio` edge cases the text leaves open — so the proven claim is "implementable from spec text + public conformance suite, without reference source," not from prose alone. The implementers' ambiguity reports converge on the same spec gaps, queued as v1.0.2 errata: `from_ratio`'s rejection surface and S-selection bracket are implicit, its RNE-to-2³¹ renormalization at S>0 is unstated, and QUANT-ACT/RMSNORM-I slice-length preconditions, empty-ARGMAX, and `rne_div` `den≤0` behavior are unspecified. Identity artifact only; dev host (i5-10210U/crosvm) named, no performance figure exists or is claimed (A31).

A NEON kernel bit-identical on real ARM silicon (A30). On 2026-08-07, the same day as A28 and A29 and one day before the clean-room test, `cis_neon::ternary_matvec_i8_neon` — using `vmulq_s8` as the ternary multiply, per-byte bit-pair extraction through a `vqtblq_s8` code LUT, exact `vpaddlq/vpadalq` widening, the shared precondition check, and the same `-128-wrap-to-scalar-fallback` contract as the x86 kernel — was proven on the GitHub `ubuntu-24.04-arm` Neoverse runner, all three suites green in one job: equivalence 5/5 (bit-identity versus the reference, including tails, code 11, extremes, and row independence), contract 6/6 (rejection surface identical to the reference on either side of the block threshold), and mechanism 3/3 (exhaustive: every byte × 4 pair positions through the LUT, all 256 activations × 4 codes through `vmulq_s8`, and widening-chain extremes). The `vmulq_s8` wrap of `-128 × -1` is pinned identical to the x86 `vpsignb` behavior in both directions by test. Both standing digest pins (A28's op-level digest and A29's token-level digest) held in the same run, and the kernel is wired into `arm-digest.yml` as a standing gate on every push. The ledger is explicit that no ARM performance number exists or is claimed here — Rule A requires named physical hardware, and this runner is shared cloud; the throughput leg stays open until an ARM board is on the bench (A30).

Token-level identity across a complete decode (A29). `cis_decode` runs the full pipeline — tokenizer, embeddings, all 7 layers, integer attention, integer LM head, `argmax_i64` — on the tracked M7 model (hidden 384, 8K vocab) and prints one digest over every token id, prompt included: `CIS_DECODE digest=67e8c0a96abc04e1 prompt_toks=4 gen_toks=64 mode=fullint`. This reproduced identically on x86_64 and aarch64 (GitHub `ubuntu-24.04` and `ubuntu-24.04-arm`, both jobs green in one run), deterministic 3/3 on the dev host; the output is coherent TinyStories text ("...there was a little girl named Lily...") — identical *and* fluent. Both digests are standing CI gates in `arm-digest.yml`: the x86 job pins the tree, the ARM job pins the ISA, so a future mismatch is attributable on sight. The pin's design choices are stated in the row and repeated here rather than left implicit: greedy decoding only, EOS ignored (a fixed 64 tokens, with no stop-condition dependence), and `CisMode::FullInt` only — the Hybrid arms are explicitly x86-only, because their f32 legs carry no cross-ISA claim. Not established: BitNet-2B scale — the 2B weights are not in the repository and a 2B decode does not fit in CI; that leg needs a local ARM board or a paid runner and stays open (A29).

Token-level identity reaches production scale, x86 only (A36). `cis_decode` run on the pruned BitNet-2B artifacts (30 layers, hidden 2560, 50,256-token vocabulary) in the `FullInt` configuration prints `CIS_DECODE digest=cab11400d737ac4a prompt_toks=4 gen_toks=64 mode=fullint`, identical across two sequential runs on the same host (i5-10210U crosvm). The artifact is the A21/A35 model with `__metadata__.aegis_config` added by `aegis-forge/add_aegis_config.py`; its tensor data blob is byte-identical to the A21/A35 model, and the same log reproduces the A21/A35 float, hybrid, and full-integer perplexity figures exactly on this artifact, so the digest and those figures share one model. The generated text is coherent English ("...in a small town called Greenfield, there lived a young girl named Lily..."). This is the x86 anchor for the BitNet-2B cross-ISA leg — the 2B counterpart of A29 — but only the x86 leg exists so far: the aarch64 run has not been performed, so the cross-ISA claim A29 makes for the M7 model does not yet extend to BitNet-2B. Identity evidence only; no timing is quoted or quotable from it (A36).

5. The decode receipt and bare-metal verification

This section, like §4, carries no timing or throughput number; every figure below is an identity claim (Rule A).

Receipt format (A32). The golden receipt `tests/golden/witness_v1_m7_once64.receipt` binds SHA-256 hashes of all three model artifacts, the 64 generated token ids, the token digest `67e8c0a96abc04e1`, and a chained SHA-256 commitment over every decode step's full i64 logit vector — chain `ae25b770bd7b22e...`. It was minted on the crosvm dev host (i5-10210U). This is not a summary digest: it is the entire integer state sequence, 64 steps deep, each step's complete logit vector folded into the chain. Every field the receipt binds is listed above exactly as the ledger row lists it — the three artifact hashes, the token ids, the token digest, and the step-by-step logit chain — because the receipt's value is that a verifier can recompute each of these independently and compare, not that it asserts a conclusion.

Verification crosses the ISA boundary in public CI (A32). On 2026-08-08, the same receipt was replayed from source on a GitHub `ubuntu-24.04-arm` hosted runner, public run `31249589879`, snapshot `ce93bbb: artifacts 3/3`, the token digest and chain `aee25b770bd7b22e...` reproduced exactly, and the harness printed `VERIFY PASS` — replay reproduced 64 tokens, the token digest, and the full logit chain `bit-for-bit`. The same run's x86-64 job verifies the identical receipt, and `arm-digest.yml` now pins both legs on every push. This upgrades the op-level digest's cross-ISA claim (§4, A28) to a full decode trajectory — 64 steps times the complete logit vector, hash-chained — rather than a single summary value. As with the other cloud-runner legs, the machine is named as precisely as the platform allows; this is an identity artifact only, and no timing is quoted or quotable from it (A32).

Physical iron, no operating system: the Dell leg (A33). On 2026-08-08 the Provable AI Kit stick booted on the Dell Inspiron 15 (i5-5200U, Broadwell-U) with no OS present and re-derived the golden receipt `bit-for-bit`. The operator was physically present; boot was via F12 with Secure Boot off. The firmware appended to `BOOTLOG.TXT`: `STAGE V: witness verify PASS — VERIFY PASS — this machine reproduced all 64 decode steps' full logit vectors bit-for-bit, with no OS underneath`. The receipt on the stick is md5-identical to the golden (`87c45bdd...`); the boot payload is the QEMU-proven `aegis-kit-iron.img` (`320e1918...`), and the stick's readback was verified `bit-identical` before boot. This completes the chain `crosvm-mint` → `QEMU` → `public CI x86-64` → `public CI aarch64 (A32)` → `physical iron, ring 0`. We state the attribution limitation as the ledger states it, not deferred to §8: the new `BOOTLOG` entry is attributed to the Dell because it was appended after the baked-in QEMU entry and carries a different firmware memory map (`MBED.BIN` at `0x3AC000` versus QEMU's `0x1780000`) — but verifier mode prints no `CPUID`, so this is log-structural evidence, not a hardware identifier (A33).

Physical iron, no operating system: the HP leg (A34). On the same date, the identical stick and identical receipt were booted on the HP Stream (Celeron N4020, Gemini Lake) for the first-ever unikernel boot on that machine: `VERIFY PASS`, payload unchanged since the Dell leg. `BOOTLOG.TXT` gained exactly one new entry (differed against the banked Dell-era log): `VERIFY PASS — this machine reproduced all 64 decode steps' full logit vectors bit-for-bit, with no OS underneath`. The scalar-path corollary matters: the N4020 lacks `AVX2`, so a `PASS` on it implies the `SSE2` scalar path executed, since `AVX2` would fault with `#UD` — the golden receipt has now been re-derived `bit-for-bit` through two disjoint kernel code paths on iron. The attribution limitation is stated as plainly as the ledger states it: the new entry's firmware memory map is identical to the Dell entry's, so in-log evidence does not discriminate the two boxes, and attribution of this entry rests on operator witness (A34). The row also records a firmware finding made incidentally on this leg: that the N4020's UEFI boots the unikernel at all was previously unknown before this boot (A34).

A third, standalone implementation verifies the receipt without the engine (A37). Everything above shows the receipt can be produced and replayed by machines running `aegis-core` itself, on either ISA. A receipt that only the reference engine can check is not yet useful to a third party — the verifier is what makes it useful, and it must not depend on the code being verified. `cis-verify` is a separate crate, written from the spec and the receipt format rather than as a fork of `aegis-core`: zero external runtime dependencies, no dependency on `aegis-core`, no `unsafe`, no `_std+alloc` at its core. On the same dev host, it reproduces the pinned op-level digest (`CIS_SELFTEST digest=76985613c965f643 ALL_PASS=true`), both pinned table digests (the exp LUT and RoPE constants), and the token-level decode digest (`CIS_DECODE digest=67e8c0a96abc04e1 prompt_toks=4 gen_toks=64 mode=fullint`) on first attempt, then verifies the golden receipt `tests/golden/witness_v1_m7_once64.receipt` end to end — all six checks (receipt parse, the three artifact hashes, prompt tokenization, the 64-step token-id sequence, the `cis-digest`, and the witness chain) — and prints `VERIFY PASS` in about 1.4 seconds. Its tamper tests fail by naming the corrupted field (token id, chain, model/vocab hash, or receipt parse), not by silently passing. Honest scope, stated plainly: this was an LM agent transcribing the spec with the reference source visible, not a clean-room reimplement in the sense A31 uses that term — it is evidence that the spec and the receipt format are re-implementable without the engine's SIMD/dispatch code, not an independent third-party audit (§8, A37).

Where a reviewer should push. Both physical legs share one structural gap: the verifier prints no CPU identifier, so the receipt proves what was computed, not unassisted which box computed it. For the Dell leg, a differing firmware memory map gives log-internal evidence; for the HP leg, no such internal signal exists, and the claim — including its `AVX2/SSE2` corollary — rests on the operator's physical presence at boot (A33, A34). We record this limitation here, next to the claims that need it, rather than only in the paper's limitations section.

6. Cost and quality

Every figure in this section is a physical-hardware measurement or a same-run, same-binary comparison logged under `docs/hardware_logs/`; none derives from QEMU/TCG (Rule A), and each number below carries its ledger row and machine name so it can be checked against the raw log rather than this prose (Rule B).

Quality

The all-integer forward pass costs **+0.0637% perplexity** against float on the M7 reference model (5.643085 vs. 5.639491, pinned 471-token teacher-forced heldout, i5-10210U crosvm, digest `0xBED4A17A1A5EE296`, A20) — comfortably inside the preregistered +5% kill line (78× headroom). The earlier integer-dominant *hybrid* configuration on the same model, in which attention and activations were still float, cost more: **+0.3127%** (5.657126 vs. 5.639491, digest `0x42E820C2A8A59CD6`, A19). The all-integer path is *closer* to float than the hybrid, not farther: dropping the two f32 → fixed re-entry quantizations that the hybrid path requires outweighs the Q0.15/Q1.30 table quantization that the full-integer path adds.

On the production-scale model, BitNet-2B, the measured figure is **+0.7408% perplexity** (integer 30.934140 vs. float 30.706665, i5-10210U crosvm, digest `0x24C4E510A86659D6`, A21). **This is the HYBRID path — attention still runs in f32.** Do not read +0.7408% as an all-integer number. The figure also carries caveats that make it non-comparable outside its own run: it uses a vocabulary pruned from 128,256 to 50,256 tokens (ASCII-oriented; never compared cross-tokenizer), and a 200-token evaluation window in an `<unk>`-dense region, which is not comparable either to published full-vocabulary perplexities or to this project's own longer-window anchor (1,898 tokens, 10.758). The only claim it supports is the *relative* integer-vs-float cost, measured in the same run with the same binary and window.

The complete all-integer forward pass on BitNet-2B — every op, including attention, in integer — is now measured: **+0.1239% perplexity** (full-integer 30.744724 vs. float 30.706665, teacher-forced, 199 scored tokens, same sh-identical artifacts and window as A21, i5-10210U crosvm, argmax digest `0xB274DE03F5862DB7`, A35), *closer* to float than the hybrid figure above — 40× inside the preregistered +5% kill line. Two sequential runs agreed on every computed value. Reaching that number required a fix: the unfixed binary panicked 2/2 runs (`normq: residual out of range, cis_infer.rs:313`). An env-gated trace (branch `cm/e1b-normq-trace`) localized the fault to the MLP, not attention — attention re-entry stayed exact at ≤38 bits on all 30 layers, while the ACT-I (relu²) MLP output landed on Q.20 at 52–55 bits, a genuine spec gap: spec §5.10 lacked the spec §5.6 per-vector block exponent that the hybrid boundary already carried. The fix — an RNE-rounded block exponent on the ACT-I output, degenerating to the identity at M7 ranges — is ratified as spec erratum v1.0.3 (spec §11). It changes only that one op: `CIS_SELFTEST 76985613c965f643` and `CIS_DECODE 67e8c0a96abc04e1`, the M7 conformance digests reported throughout this paper, are unchanged. The A21 caveats (pruned vocabulary, 200-token window, not cross-comparable, no timing) apply identically to this figure.

Cost

CIS-1's throughput cost was measured directly rather than assumed, on two physical machines, one binary, three interleaved captures across three shapes each, 9/9 bit-exact (A26). On the **Dell i5-5200U (Broadwell-U)**, scalar integer costs 25% against scalar float: C/B median **1.248×** (range 1.239–1.260). On the **HP N4020 (Gemini Lake / Goldmont Plus)**, integer is **4–14% faster**: C/B median **0.961×** (range 0.856–0.963). The two machines disagree in sign because the cost is a property of the microarchitecture, not of the semantics: Broadwell's two FMA units favor the float LUT+FMA path, while on Goldmont Plus, where floating point is weak, integer wins. The Dell result decomposes as **4.61× absent-SIMD × 1.248× semantics** — the earlier, larger "cost of determinism" figures this project had previously circulated (including its own) were measuring the cost of missing SIMD, not of integer semantics. A methodology control on the same run — A/B on the HP, where AVX2 is absent and arm A must fall back to arm B, so the true ratio is known in advance — measured 1.000× (0.999–1.001, n=9), which is what licenses trusting the Dell's decomposition.

At parity SIMD width, the picture sharpens further (A27). On the Dell, `cis_avx2::ternary_matvec_i8_avx2` measured **D/A median 0.340×** — **2.94× faster** than the hand-written float AVX2 kernel it replaces (range 0.331–0.439), bit-identical to the scalar reference 9/9 with 0 void; D/C median 0.061× is **16.4× over the scalar reference**. This result carries a method note that is part of its provenance, not incidental: the *first* arm-D run measured D/A 0.276× but was **rejected** — arm A's own repeated measurements drifted +58% across the run where clock arithmetic predicted +28%, while arms B and C tracked the clock to 0.1%, indicating arm D had perturbed arm A's cache state and flattered the ratio. The bench was then changed to re-measure arm A immediately after arm D on every repetition (A'); on the accepted run, A'/A median = 1.000× (D/A equals D/A' to three decimals), which is what licenses treating 0.340× as measured rather than assumed.

Engine context (not CIS-1 claims)

These figures characterize the surrounding unikernel/kernel-candidate work on the Dell i5-5200U. They are not part of the CIS-1 conformance or cross-ISA claims and are reported here only as supporting context.

The ring-0 unikernel was compared against a minimal-Linux decode path under a preregistered, paired, hands-off-boot protocol (A22): the preregistered throughput form gave **+3.6% / +9.4% / +5.1%** across three prompts (3/3), with 27/27 within-boot bit-exactness and byte-identical responses. The same protocol's precision prediction, however, **failed: P-V2-2 FAIL** — measured spread 4.7–9.6% against a predicted <3%. The bit-exactness held; the wall-time repeatability did not match the prereg's own expectation, and that failure is reported alongside the pass.

A column-skip kernel candidate measured **2.88–2.89×** faster than the incumbent on the real BitNet-2B `down_proj` activation distribution (ordered variant, byte-identical to the incumbent by construction and test — zero quality risk on adoption); a chain variant measured 2.80×. Its GMAC/s figures are stated **NOMINAL** (skipped work is counted as done; speedup is a time ratio), and it is not wired end-to-end — this is a kernel-level result only (A23).

A memory-bandwidth ceiling was measured on the same machine: peak sequential read 11.19 / 10.95 GB/s at one thread, 11.63 / 11.70 GB/s at four threads, against a ternary weight-stream pattern of 0.62 GB/s (A24). The stream figure is a scalar LUT-walk **lower bound** on the engine's streaming rate, not the engine's actual rate — the bench's own caveat — and its roughly 18× gap to peak sequential bandwidth is what motivates the column-skip work above.

Table 4 — every number in this section

Number	Metric	Machine	Row
+0.3127%	Hybrid-path PPL cost, M7 (5.657126 vs 5.639491)	i5-10210U crosvm	A19
+0.0637%	Full-integer PPL cost, M7 (5.643085 vs 5.639491)	i5-10210U crosvm	A20
+0.7408%	HYBRID -path (f32 attention) PPL cost, BitNet-2B (30.934140 vs 30.706665)	i5-10210U crosvm	A21
+0.1239%	FULL-INTEGER -path PPL cost, BitNet-2B, after v1.0.3 erratum (30.744724 vs 30.706665)	i5-10210U crosvm	A35
1.248×	Scalar integer/float throughput ratio, C/B	Dell i5-5200U (Broadwell-U)	A26
0.961×	Scalar integer/float throughput ratio, C/B	HP N4020 (Gemini Lake)	A26
4.61× × 1.248×	Absent-SIMD × semantics decomposition	Dell i5-5200U	A26
0.340× (2.94×)	AVX2 integer/float kernel ratio, D/A, parity SIMD	Dell i5-5200U	A27
0.061× (16.4×)	AVX2 integer kernel vs. scalar reference, D/C	Dell i5-5200U	A27
+3.6% / +9.4% / +5.1%	Ring-0 vs. minimal-Linux decode throughput, preregistered form	Dell i5-5200U	A22
4.7–9.6% (predicted <3%, FAIL)	P-V2-2 timing-precision spread	Dell i5-5200U	A22
2.88–2.89× / 2.80×	Column-skip kernel vs. incumbent, ordered/chain (NOMINAL GMAC/s)	Dell i5-5200U	A23
11.19–11.70 GB/s vs. 0.62 GB/s	Peak sequential read vs. ternary weight-stream (LUT-walk lower bound)	Dell i5-5200U	A24

7. Why this matters

For deployment without trust. Edge, air-gapped, and sovereign deployments run models on hardware and firmware the operator cannot fully audit and networks they cannot rely on. Today the operator's assurance that "the model we validated is the model that ran" rests on platform attestation — a measured boot chain that vouches for the software stack — plus the hope that the stack is deterministic. CIS-1 removes the hope. A conforming engine's output is a mathematical function of the artifacts and the input; the decode receipt is the evidence, and any conforming machine, including one booted from a USB stick with no operating system, can check it. Platform attestation and computational verification are complementary: one says which binary ran, the other says what that binary computed.

For audit and regulation. Regulators and incident investigators increasingly need to answer "what did the system output for this input, and can you prove it?" With floating-point inference the honest answer is a distribution. With CIS-1 it is a receipt: hashes of the exact artifacts, the exact tokens, and a commitment to every intermediate logit vector, replayable by the auditor on their own hardware. The receipt format is small, the verifier is the reference implementation, and the specification is public.

For research reproducibility. Every result in this paper is either a digest reproduced in public continuous integration or a measurement logged on a named physical machine, and Table 3 says which. A reader who obtains the artifacts does not have to trust our numbers; they can print the same 64-bit values. We believe this is the correct standard for inference-engine claims and that the industry's tolerance-based "matches within epsilon" reporting has hidden real divergence for years.

For the economics of edge AI. The engine that produces these receipts boots from firmware with no operating system on decade-old laptops and re-derives the receipt for the reference model there (A33, A34); the same engine runs a 2-billion-parameter ternary model in its Linux harness on the same class of hardware (A21). The complete all-integer forward pass on that 2B model now passes its preregistered quality gate (+0.1239% perplexity against float, 40× inside the +5% kill line, A35) — though the 2B model has not yet been booted on the unikernel itself; no ledger row yet establishes that. The cost of the integer semantics is a property of the microarchitecture, not of the semantics: 25% against scalar floating point on one core, 4–14% faster on another (A26), and at parity vector width the integer AVX2 kernel is 2.94× faster than the floating-point AVX2 kernel it replaces (A27). On the commodity CPUs where it was measured, verifiability did not require specialized or trusted hardware, and did not have to cost performance.

A standing invitation. The project maintains a public falsification bounty: find any machine on which a conforming build fails to reproduce the digests and the author will pay and record the finding in the research ledger as a deliverable. We would rather buy a counterexample than defend a claim. Reviewers are invited to try before publication.

8. Limitations and what is not claimed

We state these as precisely as the ledger does, because each is where a careful reviewer should push.

Scope of the semantics. The frozen v1.0 spec fixes the data types and grids (spec §3) and lists what is not claimed (spec §10); the numeric operating envelope below is the one the headroom derivations were carried out for, recorded in ledger rows A20–A21 and the v0.3 working notes (CIS-1_SPEC_DRAFT_v0.1.md), not in the frozen text itself. CIS-1 v1.0 covers the BitNet-b1.58-style decoder block (ternary linear layers, RMSNorm, GQA attention with RoPE, squared-ReLU MLP) with hidden size 384–2560, head dimension ≤ 128 , and sequence length ≤ 512 ; headroom bounds are derived for these ranges only. Bit-identity across kernel paths is claimed for the full-integer pipeline (v0.3 and later); the earlier hybrid path, which kept attention and activations in floating point, is not claimed to be cross-path identical and is reported only for the quality comparison in §6.

The clean-room implementers were language-model agents. The two from-scratch implementations that reproduced the op-level digest (A31) were written by LLM subagents given the specification text as their sole source, with reference-source access forbidden and, as far as the harness can establish, not used. This demonstrates implementability from text; it is not an independent third-party audit, and we do not describe it as one.

The `cis-verify` reimplementation is an LM-agent transcription, not a clean-room audit. Like the two implementers behind A31, `cis-verify` (§5) was built by a language-model agent — given the specification and the receipt format as reference material, in three phases — not by an independent third party working blind to the source. This demonstrates that the spec and receipt format are re-implementable without `aegis-core`'s SIMD/dispatch code and data structures; it is not the same claim as an external auditor's clean-room verification, and we do not describe it as one (A37).

Attribution of the bare-metal boot logs. The unikernel's verifier prints no CPU identifier. The Dell log entry is attributable to that machine because its firmware memory map differs from the emulator's; the HP log entry's memory map is identical to the Dell's, so in-log evidence does not discriminate the two machines and attribution of that entry rests on the operator's physical presence (A33, A34). The scalar-path corollary — the HP processor lacks AVX2, so a pass on it implies the SSE2 path executed — depends on the same attribution.

Perplexity figures are not cross-comparable. The 2B-parameter integer-vs-float comparisons — the hybrid path (A21) and the complete all-integer path after the v1.0.3 erratum (A35) — use a vocabulary pruned from 128,256 to 50,256 tokens and a 200-token evaluation window in an out-of-vocabulary-dense region; the resulting absolute perplexities must not be compared with published full-vocabulary numbers, with this project's own longer-window anchors, or with each other across runs beyond what the shared, sha-identical artifacts and window support. Only the *relative* integer-vs-float cost, measured in the same run with the same binary and window, is the claim.

Cost figures are microarchitecture-specific. The 25% cost on Broadwell-U and the 4–14% advantage on Goldmont Plus (A26) are measurements of two machines, not a model of CPUs in general; the 2.94× AVX2 result (A27) is one machine, three shapes, three captures, with its rejected first run and the order control recorded. No token-level throughput number for the integer path exists yet, and the README says so.

Cloud-runner legs are identity evidence only. The aarch64 results (A28, A29, A30, A32) were obtained on hosted CI runners; the machine is named as precisely as the platform allows, and no timing is quoted or quotable from them.

BitNet-2B's token-level identity leg is x86-only. A36 establishes the FullInt decode digest for BitNet-2B on x86-64, identical across two runs; the aarch64 leg that A29 provides for the smaller M7 model has not yet been run at 2B scale, so the cross-ISA claim in §4 does not yet extend to BitNet-2B.

What the receipt does not do. A receipt proves that a conforming computation over the bound artifacts produced the bound outputs. It does not prove which physical machine ran it (that is platform attestation's job), does not hide the model or the prompt (it is not a zero-knowledge proof), and does not protect against an adversary who controls the artifacts and the verifier together.

Retractions. This project has previously published figures it later retracted — an unlogged throughput number and two kernel-comparison claims among them. They remain in the ledger, marked, with the reasons. Every figure in this paper carries a ledger row and a raw log path (Table 3), and the reader should treat any number without one as a description, not a measurement.

9. Reproducibility statement

Repository. `https://github.com/Aefinity-AI/alice-aegis`. The spec is `docs/CIS-1_SPEC_v1.0.md`; the research ledger with every measurement's raw log path is `program/RESEARCH_LEDGER.md`.

CI workflows. `.github/workflows/arm-digest.yml` re-proves both conformance digests and the decode receipt on x86-64 and aarch64 on every push — the standing gate behind every identity claim in §4–5. This is a correctness/identity gate only (Rule A): no timing figure is ever recorded from it. `.github/workflows/aefinity-ci.yml` is the general build/format/lint gate (host job) plus an OVMF boot-correctness job for the unikernel.

Golden receipt. `tests/golden/witness_v1_m7_once64.receipt`, minted on the i5-10210U crosvm dev host and verified bit-for-bit on aarch64 in public CI (run 31249589879, snapshot `ce93bbb`, A32).

The two conformance digests. - Op-level: `CIS_SELFTEST digest=76985613c965f643 ALL_PASS=true` - Token-level decode: `CIS_DECODE digest=67e8c0a96abc04e1 prompt_toks=4 gen_toks=64 mode=fullint`

Commands (the invocation cores of `.github/workflows/arm-digest.yml`; the workflow additionally pipes each output through `grep '^CIS_...' | tee /dev/stderr | grep -q <digest> || exit 1` so a mismatch fails the job, and re-declares the artifact directory `M` before the witness step — see the file for the exact wrappers — which runs them on every push on both `ubuntu-24.04` and `ubuntu-24.04-arm`):

```
cargo build --release --example cis_selftest --manifest-path aegis-linux/Cargo.toml
./aegis-linux/target/release/examples/cis_selftest

cargo build --release --example cis_decode --manifest-path aegis-linux/Cargo.toml
M=model-lab/tinybit/m7_final_gate_work/artifacts
./aegis-linux/target/release/examples/cis_decode "$M/MODEL.SAF" "$M/EMBED.BIN" "$M/VOCAB.BIN"
64 "Once upon a time"

cargo build --release --example cis_witness --manifest-path aegis-linux/Cargo.toml
./aegis-linux/target/release/examples/cis_witness verify \
"$M/MODEL.SAF" "$M/EMBED.BIN" "$M/VOCAB.BIN" \
tests/golden/witness_v1_m7_once64.receipt
```

The first two lines reproduce the op-level digest; the next two, the token-level digest against the in-repo M7 model; the last two replay and verify the x86-minted golden receipt. A mismatch in any digest is a falsification of the corresponding claim in §4–5, not a bug to be quietly fixed — the workflow says so at the point it would fail.

Standalone verifier (cis-verify). A separate crate at `cis-verify/`, with zero external runtime dependencies and no dependency on `aegis-core` (§5, A37):

```
cargo test --features std --manifest-path cis-verify/Cargo.toml -- --nocapture

cargo run --release --features std --manifest-path cis-verify/Cargo.toml --bin cis-verify -- \
  tests/golden/witness_v1_m7_once64.receipt \
  "$M/MODEL.SAF" "$M/EMBED.BIN" "$M/VOCAB.BIN"
```

The first command runs the crate's test suite, including the pinned op-level digest, both pinned table digests, the token-level decode digest, and the golden-receipt verification and tamper tests. The second runs the CLI directly against the golden receipt and prints `VERIFY PASS` (or `VERIFY FAIL (<field>)`) in about 1.4 seconds.

Append-only evidence (Rule C). `tests/golden/` and `docs/hardware_logs/` are append-only: every figure in this paper traces to a file under one of these two paths, and no existing file under either is ever edited, only added to. Both directories are shipped with the paper's release artifact so a reader can check a claim against its raw log without re-running anything, and can re-run the commands above to check it against live hardware anyway.

Table 3 — provenance

Generated by `docs/paper/gen_table3.py` from `program/RESEARCH_LEDGER.md` rows A19-A37 and `docs/CIS1_PAPER_OUTLINE.md` §4-6. Do not hand-edit this file — edit `CLAIM_MAP` in the generator script and re-run it.

Regenerate with:

```
python3 docs/paper/gen_table3.py
```

19 claims total, 19 with a verified existing primary log file.

Paper §	Claim (short)	Value	Ledger row	Machine	Provenance (path or CI run)
4	Four-implementation digest jury (HP N4020 bare iron, Dell i5-5200U bare iron x2, crosvm, QEMU/TCG)	digest 76985613c965f643, ALL_PASS=true, 4 codegen paths	A25	HP N4020 + Dell i5-5200U + crosvm i5-10210U + QEMU/TCG	docs/hardware_logs/hp_L_BOOTLOG_2026-08-02.txt
4	Digest jury crosses to a second ISA (aarch64 CI)	same digest 76985613c965f643 on aarch64	A28	GitHub ubuntu-24.04-arm (Neoverse N2)	docs/hardware_logs/cis_selftest_aarch64_github_arm_2026-08-07.log
4	Clean-room spec-only reimplementation reproduces the digest	2 implementers, 400/484 lines, distinct by md5, first-run pass	A31	i5-10210U crosvm (dev host)	docs/hardware_logs/cis_cleanroom_tier2_2026-08-08.log
4	NEON kernel bit-identical on real ARM silicon	equivalence 5/5, contract 6/6, mechanism 3/3 exhaustive	A30	GitHub ubuntu-24.04-arm (Neoverse N2)	docs/hardware_logs/cis_neon_tests_aarch64_github_arm_2026-08-07.log
4	Token-level full-pipeline decode digest identical x86_64 vs aarch64	digest 67e8c0a96abc04e1, prompt_toks=4 gen_toks=64	A29	GitHub ubuntu-24.04 + ubuntu-24.04-arm	docs/hardware_logs/cis_decode_token_crossisa_ci_2026-08-07.log
5	Decode receipt format + CI	chain aee25b770bd7b22e..., CI	A32	i5-10210U crosvm	docs/hardware_logs/witness_receipt_aarch64_ci_2026-08-08.log (run 31249589879)

Paper §	Claim (short)	Value	Ledger row	Machine	Provenance (path or CI run)
	replay verification	run 31249589879 (snapshot ce93bbb)		(mint) + GitHub ubuntu-24.04-arm + ubuntu-24.04 (verify)	
5	Physical iron verification, Dell (AVX2 path)	STAGE V VERIFY PASS, receipt md5 87c45bdd..., BOOTLOG md5 becd7cef	A33	Dell Inspiron 15 (i5-5200U, physical iron)	docs/hardware_logs/dell_i5-5200U_kit_iron_verify_bootlog_2026-08-01.txt
5	Physical iron verification, HP N4020 (SSE2 scalar path)	VERIFY PASS, BOOTLOG md5 4fb5fc8b, one new BOOTLOG entry vs Dell	A34	HP Stream (Celeron N4020, physical iron)	docs/hardware_logs/hp_n4020_kit_iron_verify_bootlog_2026-08-01.txt
6	Quality cost, hybrid path, M7	+0.3127% PPL (float 5.639491 vs int 5.657126), digest 0x42E820C2A8A59CD6	A19	i5-10210U crosvm	docs/hardware_logs/cisl_e2_int_vs_float_ppl_m7_i5-10210U_crosvm_2026-08-01.log
6	Quality cost, full-integer path, M7	+0.0637% PPL (5.643085 vs 5.639491), digest 0xBED4A17A1A5EE296	A20	i5-10210U crosvm	docs/hardware_logs/cisl_fullint_attention_ppl_m7_i5-10210U_crosvm_2026-08-01.log
6	Quality cost, integer-dominant HYBRID path (f32 attention), BitNet-2B	+0.7408% PPL (30.744724 vs 30.706665), digest 0x24C4E510A86659D6	A21	i5-10210U crosvm	docs/hardware_logs/cisl_e2_bitnet2b_int_vs_float_ppl_i5-10210U_crosvm_2026-08-01.log
6	Quality cost, FULL-INTEGGER path, BitNet-2B (after v1.0.3 erratum)	+0.1239% PPL (30.744724 vs 30.706665), argmax digest 0xB274DE03F5862DB7	A35	i5-10210U crosvm	docs/hardware_logs/cisl_fullint_ppl_bitnet2b_i5-10210U_crosvm_2026-08-27_e1c_fixed.log
6	Throughput cost by microarchitecture (C/B ratio)	Dell (Broadwell-U) 1.248x slower; HP (Gemini Lake) 0.961x (faster)	A26	Dell i5-5200U (Broadwell-U) + HP N4020 (Gemini Lake)	docs/hardware_logs/cis_vs_float_L_dell_BOOTLOG_2026-08-05.1
6	AVX2 integer kernel vs float AVX2 kernel	D/A 0.340x = 2.94x faster; D/C 0.061x = 16.4x over scalar	A27	Dell i5-5200U	docs/hardware_logs/cis_avx2_armD_ordercontrol_L_dell_BOOTLOG_2026-08-06.txt
6	Ring-0 unikernel vs minimal Linux decode throughput	+3.6% / +9.4% / +5.1% (prereg form); P-V2-2 FAIL spread 4.7-9.6%	A22	Dell i5-5200U	docs/hardware_logs/mech2_U_BOOTLOG_2026-08-01.txt
6	Column-skip kernel candidate (engine context, not a CIS-1 claim)	2.88-2.89x vs incumbent (ordered variant); 2.80x (chain variant)	A23	Dell i5-5200U	docs/hardware_logs/mech2colskip_L_dell_BOOTLOG_2026-08-01.1
6	Bandwidth ceiling (engine context, not a CIS-1 claim)	peak seq. 11.19/10.95 GB/s (1T), 11.63/11.70 GB/s (4T) vs ternary stream 0.62 GB/s	A24	Dell i5-5200U	docs/hardware_logs/mech2colskip_L_dell_BOOTLOG_2026-08-01.1
4	Token-level FULL-INTEGGER decode digest, BitNet-2B, x86-64 leg	digest cab11400d737ac4a, prompt_toks=4 gen_toks=64, identical on 2 runs, coherent text	A36	i5-10210U crosvm	docs/hardware_logs/cis_decode_bitnet2b_fullint_x86_i5-10210U_crosvm_2026-08-27.log
5	Standalone third-party verifier (no engine dependency)	CIS_SELFTEST 76985613c965f643 ALL_PASS=true; CIS_DECODE	A37	i5-10210U crosvm	docs/hardware_logs/cis_verify_standalone_tier2_tier3_golder_10210U_crosvm_2026-08-27.log

Paper §	Claim (short)	Value	Ledger row	Machine	Provenance (path or CI run)
	reproduces both digests and verifies the golden receipt	67e8c0a96abc04e1; VERIFY PASS in ~1.4s, tamper tests name the field			

Acknowledgments and reproducibility

This paper's every quantitative claim traces to a ledger row in `program/RESEARCH_LEDGER.md` and a raw log under `docs/hardware_logs/` (Table 3); nothing here is asserted without that trail, and the repository ships both directories so a reader can check a claim against its raw log without re-running anything. Repository: <https://github.com/Aefinity-AI/alice-aegis>. The frozen specification is `docs/CIS-1_SPEC_v1.0.md`; the reproduction commands and CI workflows are in §9 above.

The \$50 falsification challenge. The author maintains a standing, public bounty (`CHALLENGE.md`): find any machine on which a conforming build produces a different digest or fails witness verification, and the author will pay \$50 and record the finding in the research ledger as a deliverable in its own right, not a bug to be quietly fixed. Reviewers are invited to try before publication.